

LA GESTION DE LA MEMOIRE

Table des matières



Objectifs	3
Introduction	4
I - STRUCTURE ET ORGANISATION DES MÉMOIRES	5
1. Hiérarchie des mémoires	5
2. Adressage logique / Adressage physique	5
II - Évaluation 1	7
III - Les politiques de gestion de la mémoire avec partitions multiples contiguës	9
1. Partitions contiguës fixes	9
2. Partitions contiguës dynamiques	10
3. Partitions contiguës Siamois (Buddy system)	11
IV - Évaluation 2	12
V - Les politiques de gestion de la mémoire avec partitions multiples non contiguës	14
1. La Pagination	14
2. La segmentation	16
3. Segmentation paginée	17
4. Application	18
VI - Évaluation 3	20
Ressources annexes	22

Objectifs

À la fin de cette leçon, vous serez capable de :

- Décrire la structure et l'organisation de la mémoire centrale
- Identifier et Expliquer les politiques de gestion de la mémoire avec partitions multiples contiguës
- Identifier et Expliquer les politiques de gestion de la mémoire avec partitions multiples non contiguës

Introduction



A l'origine, la mémoire centrale était une ressource chère et de taille limitée. Elle devait être gérée avec soin. Sa taille a considérablement augmenté depuis lors, toutes fois les applications ont eu aussi des tailles de plus en plus énormes. Cela étant, elle reste toujours une ressource rare et limitée, elle doit donc être gérée de façon optimale.

La gestion de la mémoire est l'une des tâches fondamentales d'un système d'exploitation. Un programme ne peut s'exécuter que si ses instructions et ses données sont en mémoire physique. Donc, si on désire exécuter plusieurs programmes simultanément dans un ordinateur, il faudra que tous ces programmes soient chargés dans la mémoire. Le système d'exploitation devra donc allouer à chaque programme une zone de la mémoire où celui-ci sera chargé jusqu'à sa terminaison.

Le but de cette leçon est de décrire les problèmes et les méthodes de gestion de la mémoire principale.

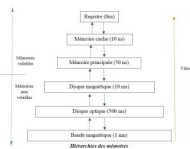
STRUCTURE ET ORGANISATION DES MÉMOIRES

I

1. Hiérarchie des mémoires

La grande variété de systèmes de stockage dans un système informatique peut être organisée dans une hiérarchie selon leur vitesse et leur coût.

- Les niveaux supérieurs sont chers mais rapides. Au fur et à mesure que nous descendons dans la hiérarchie, le coût par bit diminue alors que le temps d'accès augmente.
- En plus de la vitesse et du coût des divers systèmes de stockage, il existe aussi le problème de la volatilité de stockage. Le stockage volatile perd son contenu quand l'alimentation électrique du dispositif est coupée.



- La mémoire cache (antémémoire) : La mise en mémoire cache est un principe important des systèmes informatiques autant dans le matériel que dans le logiciel. Quand une information est utilisée, elle est copiée dans un système de stockage plus rapide, la mémoire cache, de façon temporaire. Quand on a besoin d'une information particulière, on vérifie d'abord si elle se trouve dans la mémoire cache, sinon on ira la chercher à la source et on mémorise une copie dans la mémoire cache, car on suppose qu'il existe une grande probabilité qu'on en aura besoin une autre fois.

2. Adressage logique / Adressage physique

- Mémoire physique:
 - La mémoire principale RAM de la machine
 - Adresses physiques: les adresses de cette mémoire
- Mémoire logique: l'espace d'adressage d'un programme.
- Adresses logiques: les adresses dans cet espace
- Il faut séparer ces concepts car normalement, les programmes sont chargés de fois en fois dans des positions différentes de la mémoire.
- Donc adresse physique $\neq$ adresse logique

Une adresse générée par le processeur est appelée adresse logique. Tandis qu'une adresse vue par l'unité mémoire, c'est à dire celle qui est chargée dans le registre d'adresse de la mémoire, est appelée adresse physique. Il est nécessaire au moment de l'exécution de convertir les adresses logiques en adresses physiques.

- Étapes de chargement
 - Trouver de la mémoire libre pour un module de chargement
 - Contiguë ou non contiguë
- Traduire les adresses du programme et effectuer les liaisons par rapport aux adresses où le module est chargé.
- Dans les premiers systèmes, un programme était toujours chargé dans la même zone mémoire.
- La multiprogrammation et l'allocation dynamique ont engendré le besoin de charger un programme dans différentes positions.

Évaluation 1


II

Exercice

Un programme ne peut s'exécuter que :

- ☐ si ses instructions et ses données sont sur le disque dur
- ☐ si ses instructions et ses données sont en mémoire centrale
- ☐ si ses instructions et ses données sont en mémoire virtuelle
- ☐ si ses instructions et ses données sont en mémoire ROM
- ☐ si ses instructions et ses données sont en mémoire cache

Exercice

La mémoire RAM est aussi appelée : (4 réponses)

- ☐ mémoire centrale
- ☐ mémoire physique
- ☐ mémoire principale
- ☐ mémoire vive
- ☐ antémémoire
- ☐ mémoire virtuelle
- ☐ mémoire ROM
- ☐ mémoire auxiliaire

Exercice

Parmi ces mémoire laquelle est la plus rapide

- ☐ mémoire cache
- ☐ mémoire vive
- ☐ registre

- ☐ Disque magnétique
- ☐ Disque optique

Exercice

Quelle est la rapidité par ordre croissant de ces mémoire ?

☐

Bande magnétique -> disque magnétique -> disque optique -> mémoire principale -> registre -> mémoire cache

☐

registre -> mémoire cache -> mémoire principale -> disque magnétique -> disque optique -> Bande magnétique

☐

Bande magnétique -> disque magnétique -> disque optique -> mémoire cache -> mémoire principale -> registre

☐

disque magnétique -> Bande magnétique -> disque optique -> mémoire principale -> mémoire cache -> registre

☐

Bande magnétique -> disque optique -> disque magnétique -> mémoire principale -> mémoire cache -> registre

Exercice

La mémoire logique est :

- ☐ la mémoire magnétique
- ☐ la mémoire optique
- ☐ l'espace d'adressage de la mémoire vive
- ☐ l'espace d'adressage d'un programme
- ☐ la mémoire virtuelle

Les politiques de gestion de la mémoire avec partitions multiples contiguës

III

1. Partitions contiguës fixes

Principe

Dans cette stratégie la mémoire est divisée en n partitions de tailles fixes (si possible inégales). Ce partitionnement se fait au démarrage du système. Le système d'exploitation maintient une table de description des partitions.

Cette forme d'allocation n'est plus utilisée aujourd'hui pour la mémoire centrale, mais les concepts que nous verrons sont encore utilisés pour l'allocation de fichiers sur disque.

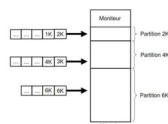
- Mémoire principale subdivisée en régions distinctes: partitions
- N'importe quel programme peut être affecté à une partition qui soit suffisamment grande
- Un programme doit tenir sur une seule partition
- Une partition ne peut contenir qu'un seul programme

Problème de fragmentation: Il y'a assez d'espace pour exécuter un programme, mais cet espace est fragmenté de façon non contiguë.

- Fragmentation interne : espace entre les partitions
- Fragmentation externe : espace dans les partitions

Méthode : Allocation de partitions fixes : files d'attente séparées

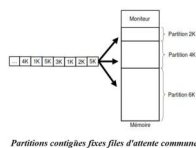
Quand un processus arrive, il peut être placé dans la file d'attente des entrées de la plus petite des partitions assez larges pour le contenir.



Partitions contiguës fixes files d'attente séparées

Méthode : Allocation de partitions fixes : file d'attente commune

Une autre organisation possible consiste à gérer une seule file d'attente. Dès qu'une partition devient libre, toute tâche placée en tête de file d'attente et dont la taille convient peut être chargée dans cette partition vide et exécutée.



2. Partitions contiguës dynamiques

Principe

Dans cette stratégie la mémoire est partitionnée dynamiquement selon la demande. Lorsqu'un processus se termine sa partition est récupérée pour être réutilisée (complètement ou partiellement) par d'autres processus.

Mais comment allouer un espace de taille n à un processus à partir d'une liste de trous libres ? On peut utiliser l'un des trois algorithmes suivant : first-fit, best-fit, worst-fit.

Le lancement des processus dans les partitions se fait selon différentes stratégies. Pour cela le gestionnaire de la mémoire doit garder trace des partitions occupées et/ou des partitions libres. On distingue les stratégies de placement suivantes:

- Stratégie du premier qui convient (First Fit): La liste des partitions libres est triée par ordre des adresses croissantes. La recherche commence par la partition libre de plus basse adresse et continue jusqu'à la rencontre de la première partition dont la taille est au moins égale à celle du processus en attente. On alloue au processus le premier bloc suffisamment grand.
- Stratégie du meilleur qui convient (Best Fit): On alloue la plus petite partition dont la taille est au moins égale à celle du processus en attente. La table des partitions libres est de préférence triée par tailles croissantes. Cet algorithme nécessite le parcours de toute la liste des espaces libres.
- Stratégie du pire qui convient (Worst Fit): On alloue au processus la partition de plus grande taille. Cet algorithme nécessite le parcours de toute la liste des espaces libres.

Des expériences ont prouvé que l'algorithme de First-fit est la meilleure.

Exemple : Allocation de partitions dynamiques contiguës

Soit une MC dont la table des partitions libres est la suivante :

Adresse	Taille
352k	32k
504k	520k

Soit les demandes suivantes P4 (24K), P5 (128K), P6 (256K) .

Evolution de la table des partitions libres:

• First Fit :

tail	P4	P5	P6
12K	1K	1K	1K
520K	520K	392K	116K

• Best Fit :

tail	P4	P5	P6
12K	1K	1K	1K
520K	520K	392K	116K

• Worst Fit :

tail	P4	P5	P6
12K	1K	12K	12K

Allocation de partitions dynamiques contiguës selon stratégies différentes

3. Partitions contiguës Siamoises (Buddy system)

C'est un compromis entre partitions de tailles fixes et partitions de tailles variables. La mémoire est allouée en unités qui sont des puissances de 2. Initialement, il existe une seule unité comprenant toute la mémoire. Lorsque de la mémoire doit être attribuée à un processus, ce dernier reçoit une unité de mémoire dont la taille est la plus petite puissance de 2 supérieure à la taille du processus. S'il n'existe aucune unité de cette taille, la plus petite unité disponible supérieure au processus est divisée en deux unités "siamoises" de la moitié de la taille de l'original. La division se poursuit jusqu'à l'obtention de la taille appropriée. De même deux unités siamoises libres sont combinées pour obtenir une unité plus grande.

Action	Mémoire			
Au départ	1Mo			
A, 150 ko	A	256ko		512ko
B, 100 ko	A	B	128ko	512ko
C, 50 ko	A	B	C	64ko
B se termine	A	128ko	C	64ko
C se termine	A	256ko		512ko
A se termine	1Mo			

(cf. p.22) (cf. p.22)

Évaluation 2



IV

Exercice

Les politiques de gestion de la mémoire avec partitions multiples contiguës sont : (4 réponses)

- ☐ Partitions contiguës Siamoisés
- ☐ Partitions non contiguës dynamiques
- ☐ Partitions contiguës dynamiques
- ☐ La partitions non contiguës fixes
- ☐ Partitions contiguës fixes avec files d'attente séparées
- ☐ Partitions contiguës fixes avec files d'attente communes
- ☐ La segmentation
- ☐ la pagination

Exercice

Les quatre principes du partitionnement contigus fixe sont :

- ☐ La mémoire virtuelle est subdivisée en régions distinctes: partitions
- ☐ La mémoire principale est subdivisée en régions distinctes: partitions
- ☐ Les programmes sont affectés à une partition qui a la même taille
- ☐ N'importe quel programme peut être affecté à une partition qui soit suffisamment grande
- ☐ Un programme doit tenir sur une seule partition
- ☐ Un programme peut tenir sur une plusieurs partitions
- ☐ Une partition peut contenir plusieurs programmes
- ☐ Une partition ne peut contenir qu'un seul programme

Exercice

La fragmentation est :

☐ un phénomène qui survient lorsqu'on ne peut plus subdiviser la mémoire

☐

un problème qui survient quand on dispose d'une partie libre de la mémoire mais qu'on ne peut pas allouer à un programme

☐ un phénomène qui survient lorsque la mémoire est saturée

☐ un phénomène qui survient lorsqu'un processus est éparpillé en mémoire

☐ un phénomène qui survient lorsque la mémoire se plante

Exercice

Les politiques de gestion de la mémoire avec partitions multiples contiguës provoquent :

☐ La fragmentation interne

☐ La défragmentation de la mémoire

☐ la volatilité de la mémoire

☐ La fragmentation externe

☐ L'éclatement de la mémoire

Exercice

Les trois stratégies de placement de processus en partitions contiguës dynamiques sont:

☐ La stratégie du premier qui convient (First Fit):

☐ La stratégie First In First Out (FIFO):

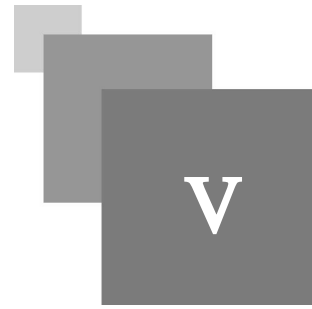
☐ Stratégie du meilleur qui convient (Best Fit)

☐ La stratégie Last In Last Out (LILO):

☐ La stratégie du pire qui convient (Worst Fit)

☐ La stratégie Tourniquet

Les politiques de gestion de la mémoire avec partitions multiples non contiguës



La traduction dynamique d'adresse réalisée par l'emploi d'un registre de base impose que les programmes soient chargés en mémoire principale dans des cellules contiguës. Nous avons vu que les techniques d'allocation associées conduisent à la fragmentation de la mémoire. Pour l'éviter, il faut pouvoir implanter un programme dans plusieurs zones non contiguës. Plusieurs techniques proposent d'allouer des espaces mémoire non-contiguës pour les processus : Pagination et Segmentation

1. La Pagination

La solution apportée par les mécanismes de pagination consiste à découper l'espace adressable, ou espace logique d'un processus en zones de taille fixe appelée pages. La mémoire réelle est également découpée en cases ou cadre (frame en anglais) ayant la taille d'une page de sorte que chaque page peut être implantée dans n'importe quelle case de la mémoire réelle.

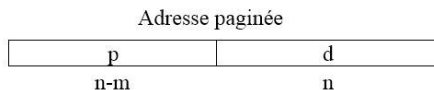
- Conséquences:
 - un processus peut être éparpillé n'importe où dans la mémoire physique.
 - la fragmentation externe est éliminée
- Principe et fonctionnement

Adressage

Une adresse est divisée en deux parties : un numéro de page p et un déplacement à l'intérieur de la page d . La taille de la page (et donc de la case) est une puissance de 2 variant généralement entre 512 et 8192 octets selon les architectures. Le choix de tailles de puissance 2 permet de simplifier la division nécessaire pour calculer le numéro de page et le déplacement à partir d'une adresse logique, ainsi :

Si la taille de l'espace d'adressage logique est 2^m et la taille d'une page est 2^n (octets ou mots):

- Les $m-n$ bits de poids fort d'une adresse paginée désignent le numéro de page ; et
- Les n bits de poids faible désignent le déplacement dans le page.



Structure d'une adresse paginée

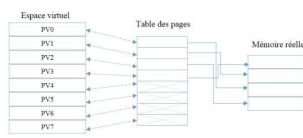
Si T est la taille d'une page et A une adresse logique, l'adresse paginée (p, d) est déduite à partir des formules :

- $p = A \text{ div } T$
- $d = A \text{ modulo } T$

L'adresse logique est égale à : $A = p * T + d$

La Table des pages

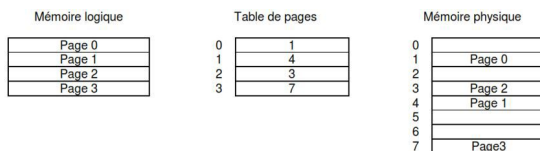
La traduction des adresses utilise une table des pages qui est située en mémoire centrale ou dans des registres, et dans laquelle les entrées successives correspondent aux pages virtuelle consécutives. La p-ième entrée de la table des pages contient le numéro r de la case où est implantée la case p, et éventuellement des indicateurs supplémentaires



Correspondance entre espace virtuel et espace réel

Exemple

L'exemple suivant montre un système ayant une mémoire logique de 4 pages et une mémoire physique de 8 pages.

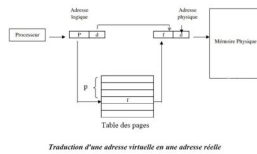


Remarque

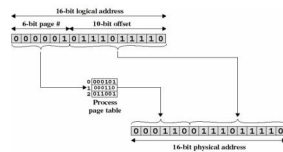
- Le SE doit maintenir une table de pages pour chaque processus.
- Chaque descripteur de pages contient le numéro de cadre où la page correspondante est physiquement localisée.
- Une table de pages est indexée par le numéro de la page afin d'obtenir le numéro du cadre.
- Avec la pagination, il n'y a plus de fragmentation externes (s'il y a suffisamment de mémoire (de pages) non allouée à un programme il les aura), par contre la fragmentation interne persiste: si un programme nécessite 2 pages et 1 octet, il aura 3 pages.

La Traduction d'adresse

L'adresse réelle (f,d) d'un mot d'adresse virtuelle (p,d) est obtenue en remplaçant le numéro de page p par le numéro de case f trouvé dans la p-ième entrée.

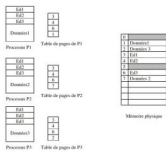


- Exemple



Page partagée

Un autre avantage de la pagination est la possibilité de partager du code commun. Cette caractéristique est particulièrement importante dans un environnement partagé. Imaginons un système qui supporte 40 utilisateurs, chacun d'eux exécutant un éditeur de textes. Si l'éditeur est constitué de 150 Ko de code et 50 Ko de données, on aura besoin de 8000 Ko pour satisfaire les 40 utilisateurs. Cependant si le code est réentrant, il peut être partagé comme le montre le schéma suivant :



Le schéma précédant montre que les 3 pages de code de l'éditeur de textes sont partagées par les 3 processus. Chaque processus, par contre, possède sa propre page de code.

Le code réentrant (encore appelé code pur) est un code invariant ; c'est à dire qu'il ne change jamais pendant l'exécution. Ainsi, deux ou plusieurs processus peuvent exécuter le même code en même temps. Chaque processus possède sa propre copie de registres et son stockage de données. On doit maintenir en mémoire physique une seule copie de l'éditeur. La table de pages de chaque processus correspond à la même structure physique de l'éditeur, mais les cadres de pages sont transformés à des cadres de pages différents. Ainsi, pour supporter 40 utilisateurs nous n'avons besoin que d'une copie de l'éditeur (ie 150 Ko), plus 40 copies de l'espace de données de 50 Ko par utilisateur. L'espace requis est donc de 2150 Ko au lieu de 8000 Ko.

2. La segmentation

La segmentation est analogue à la pagination sauf que la taille d'un segment est variable.

L'espace d'adressage logique est divisé en un ensemble de segments.

L'avantage de la segmentation par rapport à la pagination, est que les segments peuvent refléter une vision logique du programme.

- Principe

- Table de segments

La segmentation consiste à découper l'espace logique en un ensemble de segments. Chacun des segments est de longueur variable ; la taille dépend de la nature du segment dans le programme.

Une adresse logique est dite segmentée. Elle comprend un numéro de segment (s) et un déplacement dans le segment (d). (s) est utilisé comme index dans la table de segments. La table de segments est généralement implantée par des registres rapides. Si $d > \text{limite}$: alors erreur de débordement (fameux Segmentation fault). Pour cibler une zone mémoire spécifique, on doit donc désigner deux paramètres : un numéro de segment et un déplacement. :

Numéro segment	déplacement
----------------	-------------

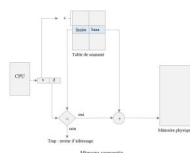
De ce point de vue, la segmentation est différente de la pagination, puisque dans la pagination, l'utilisateur ne spécifie qu'une seule adresse qui est décodée par le SE en un numéro de page et un déplacement.

En segmentation la conversion d'une adresse logique en une adresse physique est faite grâce à une table de segments. Chaque entrée de la table de segment possède deux valeurs :

- La base : c'est l'adresse de début du segment en mémoire.
- La limite : spécifie la taille du segment.

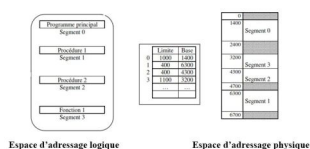
L'espace d'adressage physique correspondant peut ne pas être contigu.

Traduction d'adresse



Illustration

Soit un espace d'adressage logique contenant quatre segments



Faisons la conversion des adresses logiques suivantes en adresses physiques :

Adressage logique		Adressage physique
N° de segment	Déplacement	
2	53	$4300 + 53 = 4353$
3	852	$3200 + 852 = 4052$
0	1222	Provoque un déroulement vu que le déplacement est hors limite

- Remarque : Segmentation et fragmentation externe

Avec la segmentation la taille des segments est variable ce qui peut engendrer une fragmentation externe. Ceci peut être résolu en combinant la segmentation et la pagination.

3. Segmentation paginée

C'est une combinaison de la segmentation et de la pagination

- Adressage en segmentation paginée

L'adressage est composé de deux niveaux :

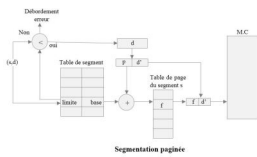
- Le niveau segment
- Le niveau page

Chaque segment est un espace linéaire d'adressage logique. Il est découpé en pages. Une adresse est formée de trois parties : (s, p, d')

- s : numéro du segment
- p : numéro de page
- d' : déplacement dans la page

Méthode : Table de page

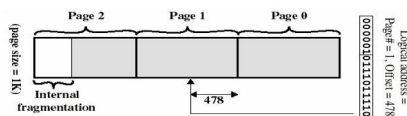
La table de page est divisée en portions. Chaque portion concerne un segment bien précis. La base permet de retrouver la portion concernant le segment donné, et p donne le numéro de page dans cette portion.



4. Application

Exemple 1

- Si 16 bits sont utilisés pour les adresses et que la taille d'une page = 1K: on a besoin de 10 bits pour le décalage, laissant ainsi 6 bits pour le numéro de page
- L'adresse logique (p,d) est traduite en adresse physique (f,d) en utilisant p comme index sur la table des pages et en la remplaçant par l'adresse f trouvée depuis la table. d ne change pas.



Exemple 2

- Hypothèses:
 - Bus d'adresses de 16 bits
 - Taille d'une page 4Ko = 4096 o (adresse sur 12bits)
- Table de page (ci-contre)
- Question : Calculer l'adresse physique correspondante à l'adresse logique 8196

15	000
14	000
13	000
12	000
11	111
10	000
9	101
8	000
7	000
6	000
5	011
4	100
3	000
2	110
1	000
0	000

- Correction

Adresse physique= 24580

Exercice 3

Soit un ordinateur ayant :

- Une mémoire physique de capacité 128 Mo
- Une architecture 32 bits.
- La taille d'une page égale à 4 Ko,
- Une adresse virtuelle indexe un octet
- Donner la taille de la table des pages.
- Chaque entrée de la table (référence d'un cadre + bit présence absence)

Correction

Taille table des pages = Taille d'une entrée de la table x Nombre d'entrées de la table

- Nombre d'entrées de la table = Taille de l'espace virtuelle / taille d'une page
- Taille de l'espace virtuelle = nombre d'adresses virtuelles possible x 1 octet = 232o (car par défaut une adresse virtuelle indexe un octet dans la mémoire virtuelle)
- Nombre d'entrées de la table = (232 octet) / (4 x 1024 octet) = 220
- La taille d'une entrée de la table des pages = 1 bit (d'absence/présence) + nombre de bits utilisés pour référencer un cadre de page dans la mémoire physique.
- Pour déterminer le nombre de ces bits il faut déterminer le nombre de cadres de page, soit égal à: $(128 \times 1024 \text{ ko}) / (4 \text{ ko}) = (27 \times 210) / (22) = 215$
- D'où le nombre de bits nécessaires pour référencer les cadres de page est 15
- taille d'une entrée de la table des pages = 1 + 15 = 16 bits
- Taille table des pages = taille d'une entrée de la table x le nombre d'entrées de la table = 220 x 16 bits = 1 Mb = 1 Mo

Évaluation 3

VI

Exercice

Les politiques de gestion de la mémoire avec partitions multiples non contiguës sont : (3 réponses)

- ☐ La pagination
- ☐ La segmentation
- ☐ l'allocation
- ☐ la dislocation
- ☐ La segmentation paginée
- ☐ la virtualisation

Exercice

La pagination consiste :

- ☐ à découper l'espace virtuel d'un processus en zones de taille variable appelée pages
- ☐ à découper la mémoire physique en zones de tailles variables appelées pages
- ☐ à découper l'espace adressable ou espace logique d'un processus en zones de taille fixe appelée pages
- ☐

La mémoire réelle est découpée en cadre suffisamment grande de sorte à être supérieur à la taille des processus

☐

La mémoire réelle est découpée en cadre ayant la taille qu'une page de sorte que chaque page peut être implantée dans n'importe quelle case de la mémoire réelle.

Exercice

Avec la pagination : (3 réponses)

- ☐ IL peut y avoir de fragmentation interne
- ☐ La fragmentation externe est éliminée
- ☐ Un processus est stocké de façon contiguë dans la mémoire physique

- ☐ Un processus peut être éparpillé n'importe où dans la mémoire physique.
- ☐ Il n'y a pas de possibilité de partager du code commun
- ☐ On a la possibilité de partager du code commun

Exercice

La segmentation consiste à :

- ☐ découper l'espace logique en un ensemble de segments de même longueur
- ☐ découper l'espace logique en un ensemble de segments de longueurs variables
- ☐ découper l'espace physique en un ensemble de segments de même longueur
- ☐ découper l'espace logique en un ensemble de pages de même taille

Exercice

La segmentation engendre

- ☐ la fragmentation interne
- ☐ la fragmentation externe
- ☐ la défragmentation de la mémoire
- ☐ le formatage de la mémoire

∇

Age Group	Percentage
18-29	15%
30-39	25%
40-49	35%
50-59	45%
60-69	55%
70-79	65%
80+	75%